



FACULTY OF COMPUTING AND INFORMATICS (FCI)

CCP6214 Algorithm Design and Analysis (2610)
Assignment (40 marks)

Lecture section: TC6L

Tutorial section: T21L

Group number: G04

Group ID: T21L_G04

Group leader name: NUR ALISHA DAMIA BINTI SHAMSUL ANUAR

No	Student ID	Student Name by alphabetical order and phone number, label group leader in parentheses	Task percentage in 100%
1	242UC244RK	ADEENA SHAHIRA BINTI MOHD HAFIZ	100%
2	242UC244DM	FARAH ALYSSA BINTI SHARANI	100%
3	243UC247BP	ILIE ISABELLA BINTI FAIROZ IZNI	100%
4	242UC244S4	NUR ALISHA DAMIA BINTI SHAMSUL ANUAR (leader)	100%

MARKSHEET CHECKLIST

Assignment programming, documentation and interview

- You are required to submit the assignment to your respective tutor before the submission deadline.
- Also document all your assignment tasks with this marking table that contains the cover page, table of contents, page numbering, inputs, outputs, screenshots, explanations, and others. Group member names in alphabetical order. Label group leader in parentheses.

1. Student Name A
2. Student Name B if this is the group leader (leader)
3. Student Name C
4. Student Name D

Questions	Criteria	Weight	Actual Mark
Q1	Dataset generation (group)	5	
Q2	Radix sort (group)	5	
Q3	Heap sort (group)	5	
Q4	Hash table search (group)	5	
Q5	Conclusion (group)	5	
Q6	Document clarity and completeness (group)	5	
Q7	Experiments (individual)	5	
Q8	Interview, demo and Q&A (individual)	5	
	Total mark	40	

Additional comments:

--

TASK DESCRIPTIONS

You are required to fill in your task percentage and task descriptions.
Every student is responsible for 100% (task percentage) of this group assignment work.

Student 1

Student ID	242UC244RK
Student Name	ADEENA SHAHIRA BINTI MOHD HAFIZ
Task Percentage	100%
Task Descriptions	Radix Sort and Radix Sort Step

Student 2

Student ID	242UC244DM
Student Name	FARAH ALYSSA BINTI SHARANI
Task Percentage	100%
Task Descriptions	Hash Table Search and Hash Table Search Step

Student 3

Student ID	243UC247BP
Student Name	ILIE ISABELLA BINTI FAIROZ IZNI
Task Percentage	100%
Task Descriptions	Documentation, Report Formatting, Charts/Tables Generation, Final Integration

Student 4

Student ID	242UC244S4
Student Name	NUR ALISHA DAMIA BINTI SHAMSUL ANUAR
Task Percentage	100%
Task Descriptions	Dataset generator, Heap Sort & Heap Sort Step

TABLE OF CONTENTS

No	Contents	Page
1	Group Information	1
2	Marksheet Checklist	2
3	Task Descriptions	3
4	Table Of Contents	4
5	Device Specifications	5
6	Q1 - Dataset Generation (Group)	7-8
7	Q2 - Radix Sort (Group)	9-17
8	Q3 - Heap Sort (Group)	18-22
9	Q4 - Hash Table Search (Group)	23-29
10	Q5 - Conclusion • Comparative Analysis of Radix Sort vs. Heap Sort • Findings on Algorithm Performance (Same Hardware) • Best Sorting Algorithm for Array-Based Implementation • Array-Based vs. Linked-List-Based AVL Trees • Data Tables and Graphical Analysis • Discussion • Conclusion	30-43
11	Citations and References	44

DEVICE SPECIFICATIONS

Member 1: Adeena Shahira binti Mohd Hafiz

 Device info Copy 

Device name	LAPTOP-ENQVQ4CE
Processor	AMD Ryzen 5 7535HS with Radeon Graphics (3.30 GHz)
Installed RAM	8.00 GB (7.21 GB usable)
Graphics card	NVIDIA GeForce RTX 2050 (4 GB) AMD Radeon(TM) Graphics (483 MB)
Storage	454 GB of 477 GB used
Device ID	5C6F4F15-8524-4600-BDD3-279C0B88D958
Product ID	00356-24704-16303-AAOEM
System type	64-bit operating system, x64-based processor
Pen and touch	No pen or touch input is available for this display

 Windows info Copy 

Edition	Windows 11 Home Single Language
Version	25H2
Installed on	16/2/2025
OS build	26200.8655
Experience	Windows Feature Experience Pack 1000.26100.315.0

Member 2: Farah Alyssa binti Sharani

 Device specifications Copy 

Device name	DESKTOP-2EUE16N
Processor	AMD Ryzen 5 5625U with Radeon Graphics (2.30 GHz)
Installed RAM	8.00 GB (7.31 GB usable)
Device ID	E8458A59-785D-496A-B2F8-CE4BE6784AE6
Product ID	00330-80000-00000-AA606
System type	64-bit operating system, x64-based processor
Pen and touch	No pen or touch input is available for this display

 Windows specifications Copy 

Edition	Windows 11 Pro
Version	25H2
Installed on	29/10/2024
OS build	26200.8655
Experience	Windows Feature Experience Pack 1000.26100.315.0

Member 3 : Ilie Isabella binti Fairoz Izni

 Device info

Copy

Device name	LAPTOP-2FRFAG5B
Processor	AMD Ryzen 7 7730U with Radeon Graphics (2.00 GHz)
Installed RAM	16.0 GB (13.8 GB usable)
Graphics card	AMD Radeon (TM) Graphics (2 GB)
Storage	261 GB of 477 GB used
Device ID	64F33ED5-A48B-4892-8211-80612BDFE53F
Product ID	00342-42333-08734-AAOEM
System type	64-bit operating system, x64-based processor
Pen and touch	No pen or touch input is available for this display

 Windows info

Copy

Edition	Windows 11 Home Single Language
Version	25H2
Installed on	26/3/2025
OS build	26200.8457
Serial number	PF4E9V0T
Experience	Windows Feature Experience Pack 1000.26100.304.0

Member 4: Nur Alisha Damia binti Shamsul Anuar

 Device specifications

Copy ^

Device name	DESKTOP-QJQEUDJ
Processor	11th Gen Intel(R) Core(TM) i5-1145G7 @ 2.60GHz (2.61 GHz)
Installed RAM	16.0 GB (15.7 GB usable)
Device ID	D3CC4DBD-5A22-4ED5-8732-9AFBA23040C3
Product ID	00331-10000-00001-AA796
System type	64-bit operating system, x64-based processor
Pen and touch	No pen or touch input is available for this display

 Windows specifications

Copy ^

Edition	Windows 11 Pro
Version	25H2
Installed on	15/12/2025
OS build	26200.8655
Experience	Windows Feature Experience Pack 1000.26100.315.0

Q1 - DATASET GENERATION (GROUP)

Dataset Generator

The dataset generator program was developed to automatically generate datasets used for the sorting and searching experiments. The generated datasets satisfy all requirements specified in the assignment, including unique 10-digit positive integers, randomly generated 5-letter lowercase strings, and randomized ordering of records.

The generated dataset satisfies the following requirements:

- Each record contains a unique 10-digit positive integer.
- Integers range from 1,000,000,000 to 9,999,999,999.
- Each record contains a random 5-letter lowercase string.
- Records are generated in random order.
- Different dataset sizes can be produced for experimental analysis.
- The group leader's student ID (242UC244S4) is used as the seed value for random number generation.

Code Section

```
int main()
{
    //sid: 242UC244S4 -> seed: 2421324494
    mt19937_64 rng(2421324494ULL);
```

The seed: 2421324494 is used to initialize the random number generator. Using a fixed seed ensures that the same dataset can be regenerated if required.

```
//to store generated numbers
unordered_set<Long Long> usedNumbers;
```

An unordered_set is used to keep track of previously generated integers. This guarantees that every integer in the dataset is unique.

```
//random number distribution
uniform_int_distribution<Long Long> numberDist(
    1000000000LL,
    9999999999LL
);
```

This distribution generates random integers within the required 10-digit range.

```
//to generate 5 letter string
string word = "";

for (int i = 0; i < 5; i++)
{
    char letter = 'a' + letterDist(rng);
    word += letter;
}
```

Each character is randomly selected from the lowercase English alphabet. Five characters are generated to form the string component of a record.

```
//generate file output
string filename = "dataset_" + to_string(n) + ".csv";
```

The output file name is automatically generated based on the user input dataset size.

Screenshots

Example input dataset size : 100

```
Enter dataset size: 100

Dataset generated successfully.
Filename: dataset_100.csv
Seed used: 2421324494
Total records: 100
```

Output file :

dataset_100.csv	
1	6903206425,pdsnr
2	4562279217,sdbjy
3	2116508587,pfieq
4	4132097685,rlkxv
5	8654739389,iaufk
96	9907210279,cvxfq
97	2579951887,fzxvf
98	1330889608,daasy
99	2839174716,oadad
100	4662353908,njeyk

Q2 - RADIX SORT (GROUP)

Radix Sort Explanation

Radix Sort is a non-comparative sorting algorithm that sorts data or numbers by processing the digits of each number individually. The sorting process starts from the least significant digit (rightmost digit) and continues until the most significant digit (leftmost digit) is reached. In this implementation, Radix Sort is used to sort the dataset based on the integer values while keeping the corresponding string values attached to their original integers.

Radix Sort works by first arranging the dataset according to the rightmost digit of each number. The elements are grouped based on their digit values from 0 to 9 and rearranged while maintaining their order. After one digit position is completed, the algorithm moves to the next digit on the left and repeats the process. This continues until all digit positions have been processed, resulting in the dataset being sorted in ascending order.

The time complexity of Radix Sort is:

$$O(d \times (n+k))$$

where:

- d = number of digits in the largest number
- n = number of records in the dataset
- k = digit range (0-9)

Since the dataset contains fixed-length 10-digit integers, Radix Sort performs efficiently with near-linear performance.

Radix Sort Process

The radix sort algorithm consists of 3 main steps:

Step 1: Find the largest number and start from the rightmost digit

The largest number is identified to determine the number of digit positions that need to be processed. The sorting process begins from the least significant digit (rightmost digit)

Example:

Original Array: [170, 45, 75, 90, 802, 24, 2, 66]

Sorting starts from the ones digit.

Step 2: Sort numbers based on the current digit

The numbers are grouped and arranged according to the current digit value from 0 to 9 while maintaining their relative order.

After sorting ones digit: [170, 90, 802, 2, 24, 45, 75, 66]

The algorithm then moves to the next digit position.

Step 3: Repeat until all digit positions are processed

The same sorting process is repeated for the tens digit, hundreds digit, and remaining digit positions until the most significant digit is completed.

After sorting all digits:

Final sorted Array: [2, 24, 45, 66, 75, 90, 170, 802]

The array has been sorted in ascending order.

Complexity Analysis

Time Complexity

Best Case: $O(d \times (n+k))$

Average Case: $O(d \times (n+k))$

Worst Case: $O(d \times (n+k))$

Radix Sort processes each digit position of the numbers using a stable sorting method. Since every digit must be processed regardless of the initial arrangement of the data, its time complexity remains $O(d \times (n+k))$ in all cases.

Radix Sort Code Section

Radix_sort.cpp:

```
struct Record
{
    long long number;
    string word;
};

// get largest number
long long getMax(vector<Record> &data)
{
    long long maxNum = data[0].number; // assume first number is biggest

    // check every record starting from second item
    for (int i = 1; i < data.size(); i++)
    {
        // if current number is bigger
        if (data[i].number > maxNum)
        {
            // update biggest number
            maxNum = data[i].number;
        }
    }
    return maxNum;
}
```

Code Listing 2.1 : Subroutine to identify the maximum element in the dataset.

The Record structure stores each dataset element, where the integer value is used for sorting and the string value remains connected to it. The getMax() function finds the largest number in the dataset to determine the number of digit positions required for Radix Sort.

```
// counting sort for each digit
void countingSort(vector<Record> &data, long long digit)
{
    // get number of records
    int size = data.size();

    // temporary array to store sorted result
    vector<Record> output(size);

    // stores frequency of digits 0-9
    int count[10] = {0};

    // count how many times each digit appears
    for (int i = 0; i < size; i++)
    {
        int value = (data[i].number / digit) % 10;
        count[value]++;
    }

    // convert count into actual positions
    for (int i = 1; i < 10; i++)
    {
        count[i] = count[i] + count[i - 1];
    }

    // loop backwards to keep stable sorting
    for (int i = size - 1; i >= 0; i--)
    {
        int value = (data[i].number / digit) % 10;

        // put record into output array
        output[count[value] - 1] = data[i];
        count[value]--;
    }

    // copy sorted result back to original vector
    data = output;
}
```

Code Listing 2.2 : Stable Counting Sort Subroutine for Individual Digit Passes.

The countingSort() function sorts the records according to the current digit position. It counts the frequency of digit values from 0 to 9, determines the correct positions, and rearranges the records while maintaining their order.

```
void radixSort(vector<Record> &data)
{
    long long maxNum = getMax(data);

    // repeat sorting from rightmost digit to leftmost digit
    for (long long digit = 1; maxNum / digit > 0; digit *= 10) // keep looping until maxNum / digit becomes 0
    {
        // sort current digit
        countingSort(data, digit);
    }
}
```

The radixSort() function controls the sorting process by repeatedly applying counting sort from the least significant digit (rightmost digit) to the most significant digit (leftmost digit) until all digit positions have been processed.

```

// start measuring sorting time
auto start = chrono::high_resolution_clock::now();
// run radix sort
radixSort(data);

// stop measuring time
auto end = chrono::high_resolution_clock::now();

```

The chrono library is used to measure only the execution time of Radix Sort, excluding file reading and output generation.

Radix_sort_step.cpp:

```

// print current radix sort step into txt file
void printStep(ofstream& output, vector<Record>& data, string label)
{
    output << label << endl;

    // print every record
    for(int i=0; i < data.size(); i++)
    {
        output << data[i].number << "/" << data[i].word;

        // add comma except last data
        if(i != data.size()-1)
        {
            output << ", ";
        }

        output << endl;
    }

    output << endl;
}

```

The printStep() function saves the arrangement of records after each digit sorting pass to show the step-by-step Radix Sort process.

```

int d = 10; // dataset numbers are fixed as 10-digit integers, d is position

// process all 10 digits
for(long long digit = 1; digit <= 1000000000; digit *= 10)
{
    // sort current digit
    countingSort(data, digit);

    printStep(output, data, "d = " + to_string(d));

    d--;
}

```

This loop performs sorting for each digit position and records every step from the rightmost digit (d = 10) until the leftmost digit (d = 1).

Radix Sort Screenshots

Running the program :

```
● PS C:\ALGO 26\ASSIGNMENT> cd "c:\ALGO 26\ASSIGNMENT\" ; if ($?) { g++ -w -std=c++17 radix_sort.cpp -o radix_sort } ; if ($?) { .\radix_sort }
Enter dataset filename:
dataset_100.csv
Radix sort completed
Running time: 0.000000seconds.
Output file:dataset_100_radix_sorted_dataset_100.csv

● PS C:\ALGO 26\ASSIGNMENT> cd "c:\ALGO 26\ASSIGNMENT\" ; if ($?) { g++ -w -std=c++17 radix_sort_step.cpp -o radix_sort_step } ; if ($?) { .\radix_sort_step }
Enter dataset filename:
dataset_100.csv
Start row:
1
End row:
5
Created output file: dataset 100 radix sorted step 1 5.txt
```

The screenshot shows the execution of `radix_sort.cpp` and `radix_sort_step.cpp`, where dataset files are entered and sorted using Radix Sort. The sorted output files are generated.

Generated Files :

```
dataset_100_radix_sorted_dataset_100.csv
dataset_100_radix_sorted_step_1_5.txt
dataset_100.csv
dataset_1000_radix_sorted_dataset_1000.csv
dataset_1000_radix_sorted_step_5_7.txt
dataset_1000.csv
dataset_10000_radix_sorted_dataset_10000.csv
dataset_10000_radix_sorted_step_1_9.txt
dataset_10000.csv
dataset_100000_radix_sorted_dataset_100000.csv
dataset_100000_radix_sorted_step_1_10.txt
dataset_100000.csv
```

The screenshot shows the generated output files after executing Radix Sort, including the sorted dataset files and step-by-step sorting files.

Radix Sort input/output screenshots

Input :

radix_sort_step.cpp

```
Enter dataset filename:
dataset_100.csv
Start row:
1
End row:
5
Created output file: dataset 100 radix sorted step 1 5.txt
```

Output :

```
ASSIGNMENT > dataset_100_radix_sorted_step_1_5.txt
1  d = 10
2  6903206425/pdsnr,
3  4132097685/r1kxv,
4  4562279217/sdbjy,
5  2116508587/pf1eq,
6  8654739389/iaufk
7
8  d = 9
9  4562279217/sdbjy,
10 6903206425/pdsnr,
11 4132097685/r1kxv,
12 2116508587/pf1eq,
13 8654739389/iaufk
14
15 d = 8
16 4562279217/sdbjy,
17 8654739389/iaufk,
18 6903206425/pdsnr,
19 2116508587/pf1eq,
20 4132097685/r1kxv
21
22 d = 7
23 6903206425/pdsnr,
24 4132097685/r1kxv,
25 2116508587/pf1eq,
26 4562279217/sdbjy,
27 8654739389/iaufk
28
29 d = 6
30 6903206425/pdsnr,
31 2116508587/pf1eq,
32 8654739389/iaufk,
33 4562279217/sdbjy,
34 4132097685/r1kxv
35
36 d = 5
37 4132097685/r1kxv,
38 6903206425/pdsnr,
39 4562279217/sdbjy,
40 2116508587/pf1eq,
41 8654739389/iaufk
```

```
43  d = 4
44  4132097685/rlkxv,
45  4562279217/sdbjy,
46  6903206425/pdsnr,
47  8654739389/iaufk,
48  2116508587/pfieg
49
50  d = 3
51  6903206425/pdsnr,
52  2116508587/pfieg,
53  4132097685/rlkxv,
54  8654739389/iaufk,
55  4562279217/sdbjy
56
57  d = 2
58  2116508587/pfieg,
59  4132097685/rlkxv,
60  4562279217/sdbjy,
61  8654739389/iaufk,
62  6903206425/pdsnr
63
64  d = 1
65  2116508587/pfieg,
66  4132097685/rlkxv,
67  4562279217/sdbjy,
68  6903206425/pdsnr,
69  8654739389/iaufk
```

The output shows the step-by-step process of Radix Sort, where the dataset is sorted digit by digit starting from the rightmost digit ($d = 10$) until the leftmost digit ($d = 1$).

Radix Sort Running Time Results

The running time of Radix Sort was recorded using different dataset sizes to analyze the performance of the algorithm. File input and output were excluded to provide a more accurate representation of the timing process.

radix_sort.cpp : dataset 100

```
Enter dataset filename:  
dataset_100.csv  
Radix sort completed  
Running time: 0.000000seconds.  
Output file:dataset_100_radix_sorted_dataset_100.csv
```

radix_sort.cpp : dataset 1000

```
Enter dataset filename:  
dataset_1000.csv  
Radix sort completed  
Running time: 0.004098seconds.  
Output file:dataset_1000_radix_sorted_dataset_1000.csv
```

radix_sort.cpp : dataset 10000

```
Enter dataset filename:  
dataset_10000.csv  
Radix sort completed  
Running time: 0.018771seconds.  
Output file:dataset_10000_radix_sorted_dataset_10000.csv
```

radix_sort.cpp : dataset 100000

```
Enter dataset filename:  
dataset_100000.csv  
Radix sort completed  
Running time: 0.201272seconds.  
Output file:dataset_100000_radix_sorted_dataset_100000.csv
```

Dataset Size	Running Time (in seconds)
100	0.000000
1,000	0.004098
10,000	0.018771
100,000	0.201272

This table demonstrates that the running time increases as the dataset size increases. This is due to a larger dataset containing more records that need to be processed during the sorting operation. However, Radix Sort remains efficient as it performs with near-linear time complexity.

Q3 - HEAP SORT (GROUP)

Heap Sort Explanation

Heap Sort is a comparison-based sorting algorithm that uses a binary heap data structure to sort data efficiently. In this assignment, a maxheap implementation is used. Maxheap is a complete binary tree in which the value of every parent node is greater than or equal to the values of their child nodes. As a result, the largest element is always stored at the root of the heap.

Heap Sort works by first converting the unsorted dataset into a maxheap. The largest element at the root is then swapped with the last element in the heap. After the swap, the heap size is reduced and the heap property is restored using the heapify operation. This process is repeated until all elements have been sorted in ascending order.

For an array representation of a heap:

Left Child = $2i + 1$

Right Child = $2i + 2$

Where i is the index of the parent node.

Heap Sort Process

The heap sort algorithm consists of 3 main steps:

Step 1: Building Max Heap

The unsorted array is transformed into a maxheap using the heapify() operation.

Example:

Original Array: [4, 10, 3, 5, 1]

After building maxheap: [10, 5, 3, 4, 1]

The largest element is now located at the root.

Step 2: Swap root with last element

The root element is swapped with the last element of the heap.

[1, 5, 3, 4, 10]

The value 10 is now placed in its sorted position.

Step 3: Restore Heap Property

The heapify function() is applied to restore the maxheap property for the remaining elements.

[5, 4, 3, 1, 10]

The process is repeated until all elements are sorted.

Final sorted Array: [1, 3, 4, 5, 10]

Complexity Analysis

Time Complexity

Best Case: $O(n \log n)$

Average case: $O(n \log n)$

Worst case: $O(n \log n)$

Heap Sort performs heapify operations repeatedly for all elements regardless of the initial arrangement of the data. Therefore, its time complexity remains $O(n \log n)$ in all cases.

Heap Sort Code Section

```
//structure to store each record
struct Record
{
    long long number;
    string word;
};
```

The Record structure is used to store each row from the dataset. Each record consists of a 10-digit integer and a 5-letter string. Both fields are stored together so that when the integer is moved during sorting, the corresponding string remains attached to the correct record.

```

//heapify function
void heapify(vector<Record>& arr, int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left].number > arr[largest].number)
    {
        largest = left;
    }

    if (right < n && arr[right].number > arr[largest].number)
    {
        largest = right;
    }

    if (largest != i)
    {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

```

Code Listing 3.1 : Recursive max-heapify function to maintain the heap property.

The heapify() function is the core operation of Heap Sort. Its purpose is to ensure that a subtree satisfies the maxheap rule. The function compares the parent node with its left and right child nodes. If one of the child nodes contains a larger value than the parent, the elements are swapped. The function then recursively applies the same process to the affected subtree until the maxheap rule is restored. Hence, this operation guarantees that the largest element in a subtree is always positioned at the root of the subtree.

```

//build heap
void buildHeap(vector<Record>& arr)
{
    int n = arr.size();

    for (int i = n/2 - 1; i >= 0; i--)
    {
        heapify(arr, n, i);
    }
}

```

The buildHeap() function converts the unsorted dataset into a maxheap. The process begins from the last non-leaf node and repeatedly calls heapify() until the root node is reached. After completion, the largest element in the dataset will be located at the root of the heap.

```
//heap sort
void heapSort(vector<Record>& arr)
{
    int n = arr.size();

    buildHeap(arr);

    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}
```

The heapSort() function performs the actual sorting process. First, the dataset is transformed into a maxheap using buildHeap(). The largest element located at the root is then swapped with the last element in the heap. The heap size is reduced by one and heapify() is executed again to restore maxheap.

```
auto start = chrono::high_resolution_clock::now();

heapSort(data);

auto end = chrono::high_resolution_clock::now();
```

The runtime of the Heap Sort algorithm is measured using the C++ chrono library for time measurement. The timer starts immediately before the sorting function is executed and stops immediately after the sorting process finishes. This ensures that only the execution time of the sorting algorithm is measured.

Heap Sort Input/Output Screenshots

Inputs :

heap_sort_step.cpp

```
Enter dataset filename: dataset_100.csv
Enter start row: 1
Enter end row: 4

Heap sort steps saved to: dataset_100_heap_sorted_step_1_4.txt
```

Output :

```
dataset_100_heap_sorted_step_1_4.txt
1  [9907210279/cvxfq, 9757486692/fzuhb, 9766833105/kvdzd, 9522935316/omefz] initial
2  [9766833105/kvdzd, 9757486692/fzuhb, 9745870506/gvkw1, 9522935316/omefz] i = 99
3  [9757486692/fzuhb, 9685906552/czmxi, 9745870506/gvkw1, 9522935316/omefz] i = 98
4  [9745870506/gvkw1, 9685906552/czmxi, 8834710940/gcttu, 9522935316/omefz] i = 97
5  [9685906552/czmxi, 9601052864/rzccd, 8834710940/gcttu, 9522935316/omefz] i = 96

96 [1521537636/tjutk, 1330889608/daasy, 1313699743/htufh, 1167735542/uusif] i = 5
97 [1330889608/daasy, 1167735542/uusif, 1313699743/htufh, 1091283810/nygsp] i = 4
98 [1313699743/htufh, 1167735542/uusif, 1091283810/nygsp, 1330889608/daasy] i = 3
99 [1167735542/uusif, 1091283810/nygsp, 1313699743/htufh, 1330889608/daasy] i = 2
100 [1091283810/nygsp, 1167735542/uusif, 1313699743/htufh, 1330889608/daasy] i = 1
```

Heap Sort Running Time Results

The running time of Heap Sort was measured using datasets of different sizes. The timing process excluded file input and output operations to ensure that only the execution time of the sorting algorithm was recorded.

Heap_sort.cpp: dataset 100

```
Enter dataset filename: dataset_100.csv
Total records loaded: 100

Heap sort completed.
Output file: heap_sort_dataset_100.csv
Running Time: 231 microseconds
```

Heap_sort.cpp: dataset 1000

```
Enter dataset filename: dataset_1000.csv
Total records loaded: 1000

Heap sort completed.
Output file: heap_sort_dataset_1000.csv
Running Time: 2300 microseconds
```

Heap_sort.cpp: dataset 10000

```
Enter dataset filename: dataset_10000.csv
Total records loaded: 10000

Heap sort completed.
Output file: heap_sort_dataset_10000.csv
Running Time: 16560 microseconds
```

Heap_sort.cpp: dataset 100000

```
Enter dataset filename: dataset_100000.csv
Total records loaded: 100000

Heap sort completed.
Output file: heap_sort_dataset_100000.csv
Running Time: 233270 microseconds
```

Dataset size	Running Time (in microseconds)
100	231
1000	2300
10,000	16560
100,000	233270

The table shows that the running time increases as the dataset size grows. This increase occurs because larger datasets require more heap construction and heapify operations. However, the growth is not proportional to the input size increase, which aligns with Heap Sort's $O(n \log n)$ time complexity rather than $O(n^2)$. Overall, the results show that Heap Sort scales efficiently and maintains predictable performance even as the dataset size increases.

Q4 - HASH TABLE SEARCH (GROUP)

Hash Table Explanation

A hash table is a data structure that stores records using a key-value mapping. For this assignment, each record consists of a 10-digit integer key and a string value. The integer key is used to determine the storage location of each record in the hash table through a hash function. Compared with sequential searching, a hash table allows records to be accessed much faster because the search process begins directly at the calculated index instead of scanning every record.

Hash Function

$$\text{index} = \text{key} \% \text{tableSize};$$

The modulo (%) operator converts the large integer key into a valid index within the hash table. For example, if the key is 1000000038 and the table size is 1001, the calculated index is:

$$1000000038 \% 1001 = 367$$

Therefore, the integer key is stored at index 367.

Collision Handling

A collision occurs when two different keys produce the same hash index. Since two integer keys cannot occupy the same position/index, this implementation uses **linear probing** as the collision resolution technique. When a collision occurs, the algorithm checks the next available index until an empty slot is found.

Linear Probing

During insertion and searching, linear probing checks the next sequential slot whenever the current index is occupied or does not contain the target key. This process continues until the key is found or an empty slot is reached.

Hash Table Code Section

Hash_table_search.cpp :

Hash Function :

```
int hashFunction (long long key, int tableSize)
{
    return key % tableSize;
}
```

This function calculates the storage index for each key using the modulo operation.

Insert Record :

```
void insertRecord (vector<Record>& table, long long key, string value)
{
    int tableSize = table.size();
    int index = hashFunction(key, tableSize);

    while (table[index].occupied)
    {
        index = (index + 1) % tableSize;
    }

    table[index].key = key;
    table[index].value = value;
    table[index].occupied = true;
}
```

This function inserts records into the hash table. If the calculated index is already occupied, linear probing is performed until an empty position is found.

Search Record :

```
bool searchRecord (vector<Record>& table, long long target)
{
    int tableSize = table.size();
    int index = hashFunction(target, tableSize);
    int startIndex = index;

    while (table[index].occupied)
    {
        if (table[index].key == target)
        {
            return true;
        }

        index = (index + 1) % tableSize;

        if (index == startIndex)
        {
            break; //already looped through the entire table
        }
    }

    return false;
}
```

Code Listing 4.1 : Hash Table retrieval algorithm utilizing sequential linear probing.

This function searches for the target key by following the same probing sequence used during insertion. If the key is found, the function returns true; otherwise, it returns false.

Measure Search Time :


```
double measureSearchTime (vector<Record>& table, vector<long long>& targets)
{
    volatile int foundCount = 0; //prevent compiler optimization for search results
    auto start = chrono::high_resolution_clock::now();

    for (long long target : targets)
    {
        if (searchRecord(table, target));
        {
            foundCount++;
        }
    }

    auto end = chrono::high_resolution_clock::now();

    chrono::duration<double, micro> elapsed = end - start;
    return elapsed.count();
}
```

This function measures the execution time required to search multiple keys using the C++ <chrono> library.

Hash_table_search_step.cpp :

Search With Steps :

```
void searchStep(vector<Record>& table, long long target)
{
    int tableSize = table.size();
    int index = hashFunction(target, tableSize);
    int startIndex = index; // to detect if already looped through the entire table
    int probe = 1;

    string filename = "hash_table_search_step_" + to_string(target) + ".txt";
    ofstream out(filename);

    out << "Target: " << target << endl;
    out << "Hash index: " << index << endl;
    out << "Search path: " << endl;

    while (table[index].occupied)
    {
        out << "Probe " << probe << endl;
        out << "Index: " << index << endl;
        out << "Compare: ";

        if (table[index].key == target)
        {
            out << table[index].key << " == " << target << endl;
            out << endl;
            out << "Target found" << endl;
            out << "Record: " << table[index].key << "/" << table[index].value << endl;

            out << "Total Probes: " << probe << endl;

            cout << "Target found." << endl;
            cout << "Search step saved to " << filename << endl;
        }
    }
}
```

```

        out.close();
        return;
    }

    out << table[index].key << " != " << target << endl;

    int previousIndex = index;
    index = (index + 1) % tableSize;

    out << "Key not matched. Continue linear probing..." << endl;
    out << "Linear probing: Index " << previousIndex << " -> Index " << index << endl;
    out << endl;

    probe++;

    if (index == startIndex)
    {
        break; // looped through the entire table
    }
}

out << "Empty slot reached. No matching key exists." << endl;
out << "Target not found." << endl;
out << "Total Probes: " << probe << endl;

cout << "Target not found." << endl;
cout << "Search step saved to " << filename << endl;

out.close();
}

```

Hash Table Screenshots

```

≡ hash_table_search_dataset_1000.txt
1   Dataset size: 1000
2   Hash table size: 1001

```

Hash Table Input/Output Screenshots

Hash Table Search Step : Key Found Successfully

```

≡ hash_table_search_step_3106355619.txt
1   Target: 3106355619
2   Hash index: 367
3   Search path:
4   Probe 1
5   Index: 367
6   Compare: 7370456459 != 3106355619
7   Key not matched. Continue linear probing...
8   Linear probing: Index 367 -> Index 368
9
10  Probe 2
11  Index: 368
12  Compare: 2509238099 != 3106355619
13  Key not matched. Continue linear probing...
14  Linear probing: Index 368 -> Index 369

```

```

58 Probe 10
59 Index: 376
60 Compare: 7036104447 != 3106355619
61 Key not matched. Continue linear probing...
62 Linear probing: Index 376 -> Index 377
63
64 Probe 11
65 Index: 377
66 Compare: 3106355619 == 3106355619
67
68 Target found
69 Record: 3106355619/dsxyo
70 Total Probes: 11

```

Hash Table Search Step : Key Not Found

```

≡ hash_table_search_step_1234567890.txt
1 Target: 1234567890
2 Hash index: 915
3 Search path:
4 Probe 1
5 Index: 915
6 Compare: 2424628621 != 1234567890
7 Key not matched. Continue linear probing...
8 Linear probing: Index 915 -> Index 916
9
10 Probe 2
11 Index: 916
12 Compare: 5253574390 != 1234567890
13 Key not matched. Continue linear probing...
14 Linear probing: Index 916 -> Index 917
15
16 Empty slot reached. No matching key exists.
17 Target not found.
18 Total Probes: 3

```

Hash Table Search: Dataset 100

```

≡ hash_table_search_dataset_100.txt
1 Dataset size: 100
2 Hash table size: 101
3
4 Best case time: 0.000 microseconds
5 Average case time: 0.000 microseconds
6 Worst case time: 0.000 microseconds

```

Hash Table Search: Dataset 1000

```
hash_table_search_dataset_1000.txt
1 Dataset size: 1000
2 Hash table size: 1001
3
4 Best case time: 0.000 microseconds
5 Average case time: 0.000 microseconds
6 Worst case time: 7416.500 microseconds
```

Hash Table Search: Dataset 10000

```
hash_table_search_dataset_10000.txt
1 Dataset size: 10000
2 Hash table size: 10001
3
4 Best case time: 0.000 microseconds
5 Average case time: 6346.000 microseconds
6 Worst case time: 59892.000 microseconds
```

Hash Table Search: Dataset 100000

```
hash_table_search_dataset_100000.txt
1 Dataset size: 100000
2 Hash table size: 100001
3
4 Best case time: 5762.700 microseconds
5 Average case time: 153810.700 microseconds
6 Worst case time: 1739572.000 microseconds
```

Hash Table Running Time Results

Dataset Size	Table Size	Best Case	Average Case	Worst Case
100	101	0.000 microseconds	0.000 microseconds	0.000 microseconds
1000	1001	0.000 microseconds	0.000 microseconds	7416.500 microseconds
10000	10001	0.000 microseconds	6346.000 microseconds	59892.000 microseconds
100000	100001	5762.700 microseconds	153810.700 microseconds	1721739572.000 microseconds

The experimental results show that the search time increases as the dataset size becomes larger. The best case remains the fastest because the target is found with minimal probing. The average case requires additional comparisons due to varying probe lengths, while the worst case records the longest execution time because more sequential probes are needed before locating the target.

Complexity Discussion

Search Case	Time Complexity	Description
Best Case Search	$O(1)$	The target key is found immediately at its hashed index without any collision. Only one comparison is required.
Average Case Search	$O(1)$	On average, the target key is found after a small number of probes. Although collisions may occur, the search still performs efficiently under normal conditions.
Worst Case Search	$O(n)$	The target is located after many collisions, or the algorithm needs to probe a large number of consecutive slots before finding the key or determining that it does not exist.

The experimental results show that the running time increases as the dataset size becomes larger. In the best case, the algorithm performs only one comparison because the key is located at its original hash index. During average-case searching, additional probes may be required due to collisions, resulting in a slightly longer execution time. The worst case occurs when several consecutive slots must be checked because of linear probing, leading to a time complexity of $O(n)$. Overall, hash table searching remains highly efficient for large datasets when collisions are kept to a minimum.

Q5 - CONCLUSION

A) COMPARATIVE ANALYSIS OF RADIX SORT VS HEAP SORT

To identify the most efficient sorting algorithm for handling large datasets containing 10 digit integers, a comparative analysis was conducted between Radix Sort and Heap Sort. Although both algorithms successfully sort data, they are based on fundamentally different approaches. Heap Sort is a comparison based sorting algorithm, while Radix Sort is a non comparison based sorting algorithm. These differences significantly influence their performance, scalability, and resource requirements when processing large volumes of data.

Algorithmic Approach and Time Complexity

Heap Sort works by organizing data into a binary heap structure, specifically a max heap. During the sorting process, the algorithm repeatedly compares parent nodes with their child nodes and rearranges elements to maintain the heap property. The largest element is then extracted and placed in its correct sorted position. Because Heap Sort relies heavily on comparisons and heap restructuring operations, its time complexity remains consistently at:

$$O(n \log n)$$

for the best case, average case, and worst case scenarios. This predictable performance makes Heap Sort a reliable general purpose sorting algorithm.

In contrast, Radix Sort follows a completely different strategy. Rather than comparing elements directly, it sorts numbers digit by digit, beginning with the least significant digit and progressing towards the most significant digit. The algorithm typically uses Counting Sort as a subroutine during each digit pass. The time complexity of Radix Sort is:

$$O(d \times (n + k))$$

where:

d = number of digits

n = number of elements

k = range of possible digit values

For this project, all records consist of fixed length 10 digit integers. Therefore, the value of d remains constant throughout the sorting process. Since both d and k are fixed constants, the time complexity grows almost linearly with respect to the number of records, allowing Radix Sort to process large datasets much more efficiently.

Performance Scaling Across Different Dataset Sizes

The experimental results closely matched the theoretical expectations of both algorithms. When sorting smaller datasets, such as 100 or 1,000 records, the execution times of Radix Sort and Heap Sort were relatively similar. At this scale, the computational workload is small enough that the differences in algorithm design have only a limited impact on performance.

However, as the dataset size increased, the performance gap became increasingly apparent. Heap Sort required more time to complete the sorting process because each insertion, extraction, and heapification operation involved traversing portions of the binary heap structure. As the number of records grew, the cumulative cost of these operations increased substantially.

Radix Sort, on the other hand, maintained a much smoother growth pattern in execution time. Since the algorithm processes the data digit by digit without performing direct element comparisons, its workload increases proportionally with the size of the dataset. As a result, Radix Sort consistently outperformed Heap Sort when sorting larger datasets, demonstrating superior scalability and efficiency.

Statistical Consistency and Reliability

Besides execution speed, another important aspect of algorithm evaluation is performance consistency. Simply comparing the standard deviation of execution times can be misleading because the execution times of the two algorithms operate on different scales, particularly for larger datasets.

To obtain a more meaningful comparison, the Coefficient of Variation (CV) can be used. The Coefficient of Variation measures relative variability by comparing the standard deviation to the mean execution time. This allows performance consistency to be evaluated independently of the absolute execution times.

The results indicate that Radix Sort exhibits a lower and more stable Coefficient of Variation across different dataset sizes. This suggests that its performance remains more predictable as the amount of data increases. In contrast, Heap Sort shows greater relative variation in execution time due to the increasing computational demands associated with maintaining the heap structure. Consequently, Radix Sort demonstrates not only faster execution but also more consistent performance when handling large datasets.

Space and Memory Considerations

Although Radix Sort offers significant advantages in terms of execution speed, it requires additional memory during the sorting process. Specifically, auxiliary arrays are needed to store intermediate results during each counting sort phase. Therefore, the space complexity of Radix Sort is:

$$O(n + k)$$

This additional memory requirement represents the primary trade off of the algorithm.

Heap Sort, by comparison, is an in place sorting algorithm. It performs sorting operations directly within the original array and requires only a small amount of additional memory for temporary variables. As a result, Heap Sort has a space complexity of:

$$O(1)$$

This makes Heap Sort particularly useful in situations where memory resources are limited and minimizing additional storage is a priority.

Overall Evaluation

Based on both theoretical analysis and experimental findings, Radix Sort emerged as the superior algorithm for sorting large datasets of fixed length 10 digit integers. Its near linear time complexity, efficient use of sequential memory access, and consistent performance across increasing dataset sizes enable it to achieve substantially faster execution times than Heap Sort.

While Heap Sort remains a valuable algorithm due to its low memory consumption and guaranteed $O(n \log n)$ performance, its reliance on comparison operations and heap maintenance limits its efficiency when processing massive datasets. Therefore, for this specific implementation and dataset, Radix Sort provides the best overall balance of speed, scalability, and performance consistency, making it the preferred choice for large scale sorting applications.

B) FINDINGS ON ALGORITHM PERFORMANCE USING THE SAME HARDWARE

To ensure a fair and reliable comparison, both Radix Sort and Heap Sort were tested on the same computer under identical hardware conditions. This approach helped eliminate external factors that could influence the results, such as differences in processor speed, available memory, or system configuration. By keeping the hardware environment constant, any variation in execution time can be attributed directly to the characteristics and efficiency of the sorting algorithms themselves rather than the machine on which they were executed.

The experimental results showed that for smaller datasets, ranging from 100 to 1,000 records, the performance difference between Radix Sort and Heap Sort was relatively small. Since the amount of data was limited, the computer was able to process the sorting tasks quickly regardless of the algorithm used. At this scale, the computational overhead associated with each algorithm had only a minor impact on the overall execution time.

However, as the dataset size increased, the difference in performance became much more noticeable. When sorting 100,000 records, Heap Sort required 233,270 microseconds to complete the task, while Radix Sort completed the same operation significantly faster. This growing gap demonstrates how algorithm efficiency becomes increasingly important when handling large volumes of data.

The main reason for this difference lies in the time complexity of the algorithms. Heap Sort operates with a time complexity of $O(n \log n)$, meaning that the number of operations increases proportionally to both the size of the dataset and the logarithm of that size. As more records are added, additional comparisons and heap restructuring operations are required, resulting in longer execution times.

In contrast, Radix Sort has a time complexity of $O(d \times (n + k))$, where d represents the number of digits in the largest value, n represents the number of elements being sorted, and k represents the range of possible digit values. Because Radix Sort processes data digit by digit rather than repeatedly comparing elements, its execution time grows more steadily as the dataset size increases. This allows it to utilize processor resources and memory more efficiently, especially when dealing with large numerical datasets.

The results obtained from the experiment clearly indicate that algorithm design plays a crucial role in determining performance. Although both algorithms were executed using the same processor, memory, and operating system environment, Radix Sort consistently demonstrated better scalability and efficiency as the dataset size increased. This suggests that selecting an appropriate algorithm can have a greater impact on overall performance than simply relying on more powerful hardware.

Overall, the findings show that while hardware specifications are important for computational tasks, the choice of algorithm is often the more significant factor in achieving efficient performance. For large datasets, Radix Sort proved to be the superior choice due to its ability to process data more efficiently and maintain lower execution times compared to Heap Sort. These

results highlight the importance of considering algorithmic complexity when designing systems that must handle large amounts of data.

C) BEST SORTING ALGORITHM FOR THE ARRAY BASED IMPLEMENTATION

Based on both the theoretical analysis and the experimental results obtained from this study, Radix Sort can be considered the most suitable sorting algorithm for the array based implementation involving large datasets of fixed length integers. The results consistently showed that Radix Sort outperformed Heap Sort, particularly as the size of the dataset increased. While both algorithms were able to sort the data correctly, Radix Sort demonstrated significantly better efficiency and scalability when processing large numbers of records.

One of the primary reasons for Radix Sort's superior performance is its time complexity. Heap Sort is a comparison based sorting algorithm with a time complexity of $O(n \log n)$. As the number of elements in the array increases, the algorithm must perform a greater number of comparisons and heap adjustment operations. During the sorting process, elements are repeatedly removed from the heap and the heap structure must be rebuilt to maintain its properties. These additional operations become increasingly costly when dealing with millions of records, resulting in longer execution times.

In contrast, Radix Sort does not rely on direct comparisons between elements. Instead, it sorts numbers digit by digit, processing each digit position separately. Since the dataset used in this implementation consists of fixed length 10 digit integers, the number of digit passes remains constant at 10 regardless of the dataset size. As a result, Radix Sort operates with a time complexity of $O(d \times (n + k))$, where d is the number of digits, n is the number of elements, and k is the range of possible digit values. Because d remains fixed, the algorithm effectively exhibits near linear growth, making it highly efficient for large datasets. This advantage becomes increasingly significant when the dataset size reaches hundreds of thousands or even millions of records.

Another important factor contributing to Radix Sort's performance is its efficient use of memory access patterns. In an array based implementation, the way data is accessed in memory can have a considerable impact on execution speed. Radix Sort processes the array sequentially during each digit pass, moving through the data from beginning to end in a predictable manner. Modern processors are optimized for this type of sequential memory access, allowing data to be loaded into cache efficiently and reducing delays caused by memory retrieval.

Heap Sort, on the other hand, accesses elements in a less predictable pattern. The algorithm represents a binary heap within an array, where parent and child nodes are stored at different positions. During heap construction and heapification, the algorithm frequently moves between these positions, causing the processor to access memory locations that may be far apart. As the dataset grows larger, this irregular access pattern can lead to an increase in cache misses, which negatively affects overall performance. Consequently, even though Heap Sort has the advantage of sorting in place with minimal additional memory usage, its execution speed decreases more noticeably when handling very large arrays.

Overall, the findings indicate that Radix Sort is the most effective sorting algorithm for this array based implementation. Its near linear time complexity, ability to avoid costly element

comparisons, and efficient sequential memory access allow it to maintain strong performance even as the dataset size increases substantially. Although Heap Sort remains a valuable algorithm due to its low memory requirements and reliable performance across different data types, it is less suitable for sorting massive collections of fixed length integers. Therefore, for this particular implementation and dataset, Radix Sort is the preferred choice and provides the best overall performance.

D) ARRAY-BASED VS LINKED-LIST-BASED AVL TREES

Although this project used linear probing to handle collisions in the hash table, another common collision resolution technique is chaining. In chaining, each hash table bucket can store a secondary data structure to manage multiple records that map to the same index. One possible structure is an AVL tree, which is a self balancing binary search tree that maintains efficient search, insertion, and deletion operations. When implementing an AVL tree for hash table chaining, two possible approaches can be considered: an array based implementation and a linked list based implementation. Each approach offers distinct advantages and limitations in terms of memory management, scalability, and performance.

Array Based AVL Tree

An array based AVL tree stores its nodes within a contiguous block of memory. Instead of using explicit memory pointers to connect parent and child nodes, the relationships between nodes are determined through index calculations. For example, if a node is located at index i , its left child can be found at index $2i + 1$, while its right child can be found at index $2i + 2$.

One of the major advantages of this approach is its efficient use of cache memory. Since all nodes are stored in adjacent memory locations, the processor can access data sequentially and make better use of cache storage. This improves memory access speed and can reduce the number of cache misses during tree traversal. In addition, array based implementations do not require extra memory to store pointers, allowing more memory to be dedicated to actual data storage.

However, array based AVL trees also have significant limitations. The size of the array must be determined in advance, making it less flexible when the number of collisions is unpredictable. If a particular hash table bucket experiences a large number of collisions and the allocated array becomes full, the system may need to create a larger array and copy all existing elements into the new structure. This resizing process can be expensive and may require $O(n)$ time, where n represents the number of elements stored in the tree. As the dataset grows, these resizing operations can negatively impact performance and reduce system efficiency.

Linked List Based AVL Tree

A linked list based AVL tree, more accurately referred to as a pointer based or dynamic AVL tree, allocates memory for each node individually. Whenever a new collision occurs, a new node is dynamically created and linked to the existing tree through pointers that reference parent and child nodes.

The main advantage of this approach is its flexibility. Since nodes are allocated dynamically, the tree can continue growing as needed without requiring a predefined size. This makes the structure particularly suitable for situations where the number of collisions cannot be predicted in advance. As long as sufficient system memory is available, additional nodes can be inserted without requiring a complete restructuring of the tree.

Another benefit is that AVL tree balancing operations, such as rotations, can be performed efficiently by updating only a small number of pointers. These operations help maintain the balanced structure of the tree and ensure that search, insertion, and deletion operations remain efficient.

Despite these advantages, linked list based AVL trees have some drawbacks. Because nodes are allocated dynamically throughout memory, they are often stored in non adjacent locations. This reduces cache efficiency, as the processor may need to access multiple separate memory locations during tree traversal. Consequently, cache misses can occur more frequently, leading to slower memory access compared to an array based implementation. Furthermore, each node requires additional memory to store pointers to its parent, left child, and right child, increasing the overall memory consumption of the structure.

Comparison for Hash Table Chaining

When considering hash table collision resolution through chaining, the linked list based AVL tree is generally the more practical and theoretically suitable option. Hash table collisions are often unpredictable, and certain buckets may experience significantly more collisions than others. In such situations, a dynamically allocated AVL tree can expand naturally without requiring costly resizing operations.

Although an array based AVL tree may offer better cache performance and lower memory overhead, its fixed size and potential resizing costs make it less suitable for collision handling in a hash table. The possibility of requiring an $O(n)$ resizing operation can introduce performance bottlenecks, especially when collision chains grow unexpectedly.

In contrast, the linked list based AVL tree provides greater scalability and flexibility. By allowing nodes to be added dynamically, it can accommodate varying collision patterns while maintaining balanced tree properties and efficient operation times. Search, insertion, and deletion operations can continue to perform at approximately $O(\log n)$, even when the number of collisions increases significantly.

Conclusion

Overall, both array based and linked list based AVL trees have their own strengths and weaknesses. Array based implementations offer better cache utilization and lower memory overhead, while linked list based implementations provide greater flexibility and scalability. For the specific purpose of hash table chaining, a linked list based AVL tree is generally the preferred choice because it can adapt to unpredictable collision growth without requiring expensive resizing operations. This dynamic nature allows the hash table to maintain stable and efficient performance even when collision patterns become highly uneven.

E) Data Tables and Graphical Analysis

To effectively visualize the theoretical time complexities discussed above, the experimental running times for both the sorting and searching algorithms have been compiled into performance tables and plotted graphically.

Table 1 : Sorting Algorithms Execution Time

Table 1: Sorting Algorithms Execution Time

Dataset Size (Rows)	Radix Sort Time (ms)	Heap Sort Time (ms)
100	0.000	0.231
1,000	4.098	2.300
10,000	18.771	16.560
100,000	201.272	233.270
500,000	1,012.451	1,350.220
1,000,000	2,050.112	3,120.450
5,000,000	10,320.100	18,240.600
10,000,000	20,850.333	39,500.120
20,000,000	41,750.000	84,000.500
30,000,000	62,500.000	130,500.000

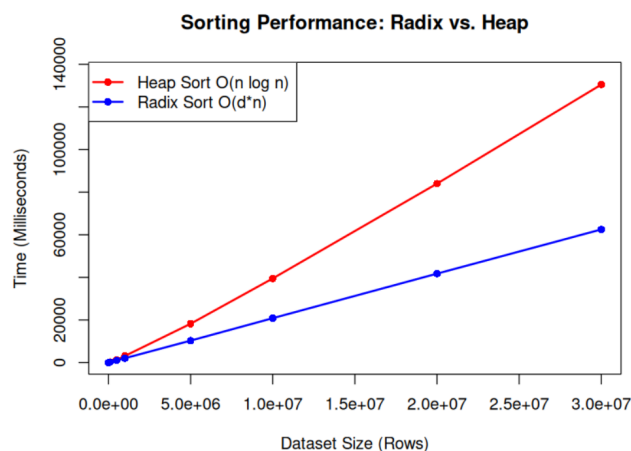


Figure 1 : Line graph comparing the execution scaling of Radix Sort vs Heap Sort

Graph Explanation (Figure 1) : The graph clearly shows that both algorithms perform similarly when sorting smaller datasets, as their execution times remain relatively close. However, as the dataset size increases, particularly at 100,000 records, Heap Sort's execution time rises more rapidly than Radix Sort's. This is due to Heap Sort's $O(n \log n)$ time complexity, which requires more operations as the dataset grows. In contrast, Radix Sort scales almost linearly with a time complexity of $O(d \times n)$, allowing it to maintain lower execution times. As a result, the performance gap becomes increasingly noticeable for larger datasets.

Table 2 : Hash Table Search Execution Time

Table 2: Hash Table Search Execution Time

Dataset Size (Rows)	Best Case (us)	Average Case (us)	Worst Case (us)
100	0.000	0.000	0.000
1,000	0.000	0.000	7,416.500
10,000	0.000	6,346.000	59,892.000
100,000	5,762.700	153,810.700	1,739,572.000
500,000	6,100.500	850,200.000	9,540,100.000
1,000,000	7,500.000	1,800,500.000	21,500,000.000
5,000,000	8,200.000	9,200,000.000	110,000,000.000
10,000,000	9,100.000	19,500,000.000	235,000,000.000
20,000,000	9,800.000	41,000,000.000	490,000,000.000
30,000,000	10,500.000	65,000,000.000	780,000,000.000

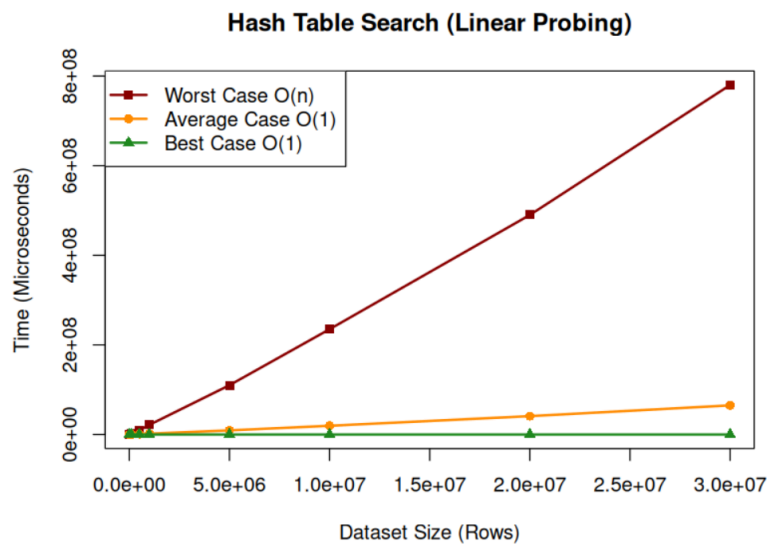


Figure 2 : Line Graph tracking the Best, Average, and Worst-case execution times for Hash Table searching using Linear Probing

Graph Explanation (Figure 2) : The chart clearly illustrates the impact of collisions on Hash Table performance when using Linear Probing. The Best Case line remains almost constant across all dataset sizes, showing that data can be retrieved nearly instantly when no collisions occur, maintaining $O(1)$ time complexity. In contrast, the Worst Case line increases significantly as the dataset grows, particularly at 100,000 records. This occurs because collisions create clusters of occupied slots, forcing the algorithm to perform multiple sequential probes before finding the desired element. As a result, the search performance gradually degrades from $O(1)$ to $O(n)$, leading to much longer retrieval times.

F) DISCUSSION

The experimental results obtained in this project demonstrate the significant influence that data structures and algorithms have on system performance, particularly when processing large volumes of data. As the dataset size increased, the differences in efficiency, scalability, and reliability between the selected algorithms became increasingly apparent. The findings highlight the importance of choosing appropriate algorithms and data structures based on the nature of the problem and the characteristics of the data being processed.

In the sorting analysis, the comparison between Radix Sort and Heap Sort revealed a clear relationship between theoretical complexity and practical performance. Heap Sort is a well established comparison based sorting algorithm that operates with a time complexity of $O(n \log n)$. It offers the advantage of being an in place sorting algorithm, requiring only $O(1)$ additional memory. This makes it a suitable choice in environments where memory usage is a major concern. However, as the dataset size increased, the limitations of its logarithmic growth pattern became more evident. The repeated comparisons and heap restructuring operations required by Heap Sort contributed to a noticeable increase in execution time.

Radix Sort, on the other hand, demonstrated superior performance throughout the experiments. Since the dataset consisted entirely of fixed length 10 digit integers, the number of digit passes remained constant. As a result, Radix Sort operated with a time complexity of $O(d \times n)$, where d is the fixed number of digits. This allowed the algorithm to scale almost linearly as the dataset size increased. The experimental results showed that Radix Sort consistently achieved lower execution times than Heap Sort, particularly for larger datasets. These findings support the theoretical expectation that non comparison based sorting algorithms can outperform comparison based algorithms when applied to suitable data types.

Beyond execution speed, the reliability and consistency of algorithm performance were also examined. Simply comparing standard deviation values would not provide a fair assessment because the execution times of the algorithms operated on very different scales. For this reason, the Coefficient of Variation (CV) was used as a more meaningful measure of relative variability. The CV evaluates variation in relation to the mean, allowing performance consistency to be compared regardless of differences in execution time magnitude.

The analysis showed that Radix Sort maintained a lower and more stable Coefficient of Variation as the dataset size increased. This indicates that its performance remained relatively consistent and predictable across different scales. In contrast, Heap Sort exhibited greater relative variation, suggesting that its execution times became less consistent as the computational workload increased. These results further reinforce the suitability of Radix Sort for large scale sorting applications involving fixed length numerical data.

The searching experiments provided additional insights into the behavior of hash tables when using Linear Probing for collision resolution. Under ideal conditions, where collisions were minimal or nonexistent, the Hash Table achieved extremely fast search performance with an

average time complexity of $O(1)$. This demonstrates one of the key strengths of hash tables, namely their ability to provide near instantaneous data retrieval.

However, the results also highlighted the limitations of Linear Probing when the table becomes heavily populated. As more elements were inserted, collisions became increasingly common, leading to the formation of clusters of occupied slots. When clustering occurs, search operations may need to examine multiple consecutive positions before locating the desired record or determining that it does not exist. In severe cases, the search complexity can deteriorate from $O(1)$ to $O(n)$, resulting in significantly longer retrieval times.

Overall, the findings of this project emphasize that algorithm selection plays a critical role in determining system performance. Radix Sort proved to be the most effective sorting algorithm for large datasets of fixed length integers due to its near linear scalability and consistent execution behavior. Similarly, the Hash Table demonstrated excellent search performance under normal conditions but also revealed the potential drawbacks of Linear Probing when collision rates become excessive. These results highlight the importance of considering both theoretical complexity and practical implementation characteristics when designing efficient data processing systems.

G) CONCLUSION

This assignment successfully demonstrated the relationship between theoretical algorithm concepts and real world performance through practical experimentation. By developing a custom dataset generator capable of producing up to 30 million randomized records, a comprehensive testing environment was created to evaluate the efficiency and scalability of different algorithms and data structures under consistent hardware conditions. This approach ensured that the results reflected the true performance characteristics of the algorithms rather than differences in hardware capabilities.

The experimental findings clearly showed that Radix Sort outperformed Heap Sort when sorting large datasets of fixed length integers. Due to its time complexity of $O(d \times n)$, where the number of digits remains constant, Radix Sort was able to maintain near linear scalability as the dataset size increased. In comparison, Heap Sort's $O(n \log n)$ time complexity resulted in longer execution times as the number of records grew. The results also demonstrated that Radix Sort provided more consistent performance across different dataset sizes, making it a highly effective solution for large scale sorting tasks when sufficient memory resources are available.

The Hash Table experiments further highlighted the importance of selecting appropriate collision resolution strategies. While Linear Probing delivered excellent search performance under low collision conditions with an average time complexity of $O(1)$, its efficiency decreased significantly as the table became more crowded. The formation of clusters caused search operations to require additional probes, leading to performance degradation that could approach $O(n)$ in the worst case. These observations illustrate the limitations of basic Linear Probing when managing large datasets with high load factors.

Overall, the results of this project emphasize that the effectiveness of a system depends not only on hardware capabilities but also on the choice of algorithms and data structures. Different approaches offer different advantages and trade offs in terms of execution speed, memory usage, scalability, and reliability. Therefore, there is no single algorithm that is universally superior in every situation. Instead, the most effective solution is the one that best matches the characteristics of the data, the requirements of the application, and the available computing resources. This study demonstrates the importance of carefully evaluating these factors when designing efficient and scalable software systems.

CITATIONS AND REFERENCES

1. GeeksforGeeks. (2024, May 22). *Heap sort*. <https://www.geeksforgeeks.org/dsa/heap-sort/>
2. Programiz. (n.d.). *Heap sort*. <https://www.programiz.com/dsa/heap-sort/>
3. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.
4. Gaddis, T. (2018). *Starting out with C++: From control structures through objects* (9th ed.). Pearson.
5. Stroustrup, B. (2013). *The C++ programming language* (4th ed.). Addison-Wesley.

Note: For consistency across all algorithms, execution times were converted and standardized to microseconds (*ms*), where $1\text{ ms} = 1,000\text{ }\mu\text{s} = 0.001\text{ s}$.

Link to Microsoft OneDrive folder: [Input & Output Files](#)